

FloodSense

Predicting Regional Flood Probability and Risk Tier from Hydro-Environmental and Infrastructure Indicators

Case Study 74: Flood Risk Analysis

B.Tech CSE (2024-28) | Semester V | Machine Learning

Submitted by: Sarthak Kanade

October 2026

This document covers the deliverables asked in the case study: problem definition, dataset and documentation, exploratory analysis, preprocessing, model development, evaluation and the Streamlit application. The code, notebooks, saved model and app are in the project folder `flood-risk/`.

Contents

1. Problem Definition
2. Dataset and Documentation
3. Exploratory Data Analysis
4. Preprocessing
5. Model Development and Experiments
6. Evaluation
7. Streamlit Application
8. Limitations
9. How to Run the Project
10. Project File Index

1. Problem Definition

A local authority wants to study flood-related data and find patterns that help predict flood risk. In this project I treat that as a supervised machine learning problem on tabular data, with two outputs:

- **Main task (regression):** predict `FloodProbability`, a continuous value that ranges from about 0.285 to 0.725 in the data.
- **Derived task (3-class classification):** convert the predicted probability into a risk tier - Low, Medium or High - because an authority takes action on risk levels, not on decimals like 0.517.

Project title: FloodSense: Predicting Regional Flood Probability and Risk Tier from Hydro-Environmental and Infrastructure Indicators.

Intended user: a municipal planning or disaster-response office that wants a quick risk screening of a region from a set of indicator scores.

Success criteria. For regression: low RMSE and MAE, and a high R^2 . For tiers: good recall on the High class, since missing a high-risk region is worse than raising a false alarm.

Scope note. The data is score-based and most likely synthetic (see Section 2), so FloodSense is a regional *risk-screening* model for academic purposes, not a flood forecasting system.

2. Dataset and Documentation

Source: Kaggle, "Flood Prediction Dataset" by naiyakhalid (kaggle.com/datasets/naiyakhalid/flood-prediction-dataset). The download contained four files:

File	Rows	Columns	How it was used
train.csv	1,117,957	id + 20 indicators + FloodProbability	Used for all modelling
flood.csv	50,000	20 indicators + FloodProbability	Used once, for the data-origin check below
test.csv	745,305	id + 20 indicators (no target)	Not used - no target to evaluate against
sample_submission.csv	745,305	id, FloodProbability	Not used - competition format file

Sampling. Using all 1.1 million rows would make cross-validation, Random Forest and tuning very slow on a laptop. I took a random sample of 100,000 rows from train.csv (`random_state=42`), dropped the id column, and saved it as `data/processed/sample_100k.csv`. Everything after this point uses that sample.

Variables

The 20 predictors are integer indicator scores (0 to 19 in train.csv). The target is `FloodProbability`.

Group	Indicators
Climate and water	MonsoonIntensity, ClimateChange, TopographyDrainage, RiverManagement, Siltation, Watersheds, CoastalVulnerability, Landslides, WetlandLoss
Infrastructure and planning	DamsQuality, DrainageSystems, DeterioratingInfrastructure, Encroachments, InadequatePlanning, IneffectiveDisasterPreparedness
Land use and society	Deforestation, Urbanization, AgriculturalPractices, PopulationScore, PoliticalFactors

Data-quality observations

- **Missing values:** none in train.csv or flood.csv.
- **Duplicate rows:** none in train.csv or flood.csv.
- **Ranges:** indicators are non-negative integers; the target stays within 0.285 to 0.725 (mean 0.5045, standard deviation 0.051 on the full train.csv).
- **Data origin check:** in flood.csv, FloodProbability equals the sum of the 20 indicators divided by 200 on every row (maximum difference 0). That makes it a fixed formula, and a plain Linear Regression scores $R^2 = 1.0$, so it cannot be used to compare models. In train.csv the same formula is *not* exact (mean absolute difference about 0.018), which shows noise was added. This is why train.csv was chosen for modelling.
- **Synthetic data:** the integer scores, the exact formula in flood.csv and the added noise strongly suggest the data is generated, not measured. I could not verify exactly how it was produced, so I describe it as noisy, likely synthetic data.

3. Exploratory Data Analysis

The full EDA with plots is in notebooks/01_eda.ipynb. The figures are saved in reports/figures/: target_distribution, indicator_distributions, correlation_heatmap, top_correlations, mean_target_by_score and outliers_boxplot. The main findings (numbers measured on the full train.csv) are:

- **Target:** centred near 0.50 with a narrow spread (standard deviation 0.051), so the model has to separate small differences.
- **Indicators look alike:** every indicator has a mean of about 5 and a standard deviation of about 2.05 to 2.09, with values from 0 to 19.
- **No standout predictor:** each of the 20 indicators correlates with the target by about the same amount, between 0.178 (CoastalVulnerability) and 0.190 (DeterioratingInfrastructure). So the signal is spread evenly across all indicators instead of coming from one or two.
- **No multicollinearity:** the largest correlation between any two indicators is only about 0.016, so the indicators are almost independent of each other and none had to be dropped.
- **Outliers:** the indicators are bounded integer scores, so high or low scores are valid values, not errors. No outliers were removed.
- **What this suggests for modelling:** an even, independent, additive contribution from every indicator points to a mostly linear relationship, which is why a plain linear model already does well (Section 5).

4. Preprocessing

Preprocessing is implemented in src/preprocess.py. Each decision and the reason:

Step	What was done	Why
Drop id	Removed the id column	It is only a row label, not a predictor
Sampling	100,000 random rows, random_state=42	Keeps training fast; seed makes it reproducible

Step	What was done	Why
Cleaning	Checked missing values and duplicates (none found)	Nothing needed to be removed or imputed
Train/test split	80/20 split before any fitting: 80,000 train, 20,000 test	Keeps the test set unseen, avoids data leakage
Scaling	StandardScaler inside a scikit-learn Pipeline, fitted on training data only; used for Linear, Ridge and MLP, not for tree models	Linear and neural models are sensitive to scale; trees are not. Pipeline prevents leakage
Risk tiers	Cutoffs from the tertiles of the target: Low up to 0.48, Medium 0.48 to 0.525, High above 0.525 (saved in models/tier_cutoffs.json)	Gives roughly equal-sized groups, no arbitrary thresholds
Feature engineering	Three simple aggregate features added to the 20 indicators (23 features), built inside the model pipeline	Tested as one experiment; being inside the pipeline means the app needs only the 20 raw inputs

5. Model Development and Experiments

Models were compared with 5-fold cross-validation on the training set (code in `src/train.py`, notebook `02_modeling.ipynb`). In total 13 configurations were run; the full table is in `reports/results/model_comparison.csv`. The main ones are below.

Model	Experiment type	CV R^2	Test R^2	Test RMSE	Test MAE
HistGradientBoosting + 3 aggregates (23 features)	Feature engineering	0.86503	0.87092	0.01845	0.01441
MLP (scaled, 32-16 layers)	Neural network	0.84457	0.85215	0.01974	0.01554
Linear Regression (scaled)	Baseline	0.84421	0.84958	0.01991	0.01570
Ridge (alpha 1.0)	Regularisation	0.84421	0.84958	0.01991	0.01570
Tuned Ridge (alpha 0.207)	Hyperparameter tuning	0.84421	0.84958	0.01991	0.01570
Tuned HistGradientBoosting (20 features)	Hyperparameter tuning	0.79760	0.80573	0.02263	0.01809
HistGradientBoosting default (20 features)	Ensemble	0.75598	0.76422	0.02493	0.02039
Random Forest (50 trees, max_depth 10)	Ensemble	0.43933	0.43646	0.03854	0.03150

Reading the results

- **The linear baseline is strong** (R^2 about 0.85). Ridge and the tuned Ridge give exactly the same score because with 80,000 rows and 20 features, regularisation has almost nothing to fix.
- **Tree models on the 20 raw indicators did worse than linear.** The target depends on a sum-like combination of all 20 indicators, which is easy for a linear model but hard for trees, since each split looks at one indicator at a time. The Random Forest was also limited to depth 10 and 50 trees, which probably explains its low score; this result is for that restricted setting, not for Random Forests in general.
- **Tuning helped the boosting model** (CV R^2 0.756 to 0.798) but not the linear models.

- **Aggregate features gave the best result** (CV R^2 0.865). An aggregate such as an average of the indicators carries the additive signal directly, so the tree model no longer has to build it split by split. These features do not add new information - they are combinations of existing columns - they only make the pattern easier for trees to learn.
- **Final model:** HistGradientBoosting with the 3 aggregate features, chosen because it has the best cross-validation score. The test score (0.871) is close to the CV score (0.865), so there is no sign of overfitting.

Note on the gap: the best model beats the linear baseline by about 0.02 in R^2 . That is a real but modest gain, which fits the finding that the signal is mostly additive.

6. Evaluation

The final model was evaluated once on the unseen test set of 20,000 rows (`src/evaluate.py`, results saved in `models/metrics.json`).

Measure	Value	Meaning
R^2	0.87092	The model explains about 87% of the variation in flood probability
RMSE	0.01845	Typical prediction error is under 0.02 on a 0 to 1 scale
MAE	0.01441	Average absolute error is about 0.014
High-risk recall	90.51%	About 9 in 10 truly high-risk regions are caught as High
High-risk labelled Low	0	No true High region was predicted as Low
Macro-averaged F1	0.73299	Average F1 over the three tiers

Why these measures

RMSE and MAE show the error in the same units as the probability, and R^2 compares the model with simply predicting the average. For the tiers, recall on the High class matters most because the cost of missing a high-risk region is higher than a false alarm; macro-F1 treats all three tiers equally. Accuracy alone would hide weak performance on one tier.

Diagnostic plots

Saved in `reports/figures/`: `confusion_matrix`, `predicted_vs_actual`, `residuals_vs_predicted`, `residual_histogram` and `permutation_importance`.

To add from your own outputs before submitting: (1) one or two sentences on what the residual plots and the largest errors show (for example, whether errors are bigger near the tier boundaries); (2) the top indicators from your permutation importance plot and whether they make sense. I did not have those plots, so I have not written conclusions about them.

7. Streamlit Application

The app is in `app/streamlit_app.py`. It loads the saved pipeline and the tier cutoffs; it does not retrain anything.

Tab	What the user sees
Regional Risk Assessment	20 sliders, one per indicator (set by the user). The app shows the predicted flood probability and a colour-coded risk tier (Low, Medium, High).

Tab	What the user sees
Model Performance and Audit	Key metrics, the model comparison table and the diagnostic plots, loaded from the saved files.

Because the feature engineering lives inside the saved pipeline, the user only has to enter the 20 raw indicator values. Run it with `PYTHONPATH=. streamlit run app/streamlit_app.py` and open `http://localhost:8501`.

Add: one or two screenshots of the app (a Low-risk and a High-risk input) in this section for your submission and viva evidence.

8. Limitations

- **Synthetic, score-based data.** Indicators are integer scores and the data was likely generated, so results show how the model learns this dataset, not how it would behave on real measurements.
- **No geography or time.** There are no coordinates or dates, so the model cannot say where or when a flood happens. It is a screening tool only.
- **Mostly additive signal.** The linear baseline is already strong, so complex models give a modest gain (about 0.02 R^2).
- **Relative tiers.** Tier cutoffs are tertiles of this dataset, not physical flood thresholds. A real authority would need to set cutoffs from real outcomes.
- **Sampling.** Only 100,000 of 1.1 million rows were used; scores could shift slightly with more data or a different sample.
- **Limited tuning.** The Random Forest used a restricted setting (50 trees, depth 10) and tuning was small, so the tree-model results are not their best possible.
- **Correlation, not cause.** The model finds patterns in the scores; it does not prove that changing an indicator would change flood risk.

With real data, I would add location and date, validate on a different region than the one used for training, and set tier thresholds together with flood-management experts.

9. How to Run the Project

From inside the `flood-risk` folder:

```
python3 -m venv venv
source venv/bin/activate (Windows: venv\Scripts\activate)
pip install -r requirements.txt
```

Then run in this order:

Step	Command	Produces
1. EDA	<code>jupyter notebook notebooks/01_eda.ipynb</code>	EDA figures in <code>reports/figures/</code>
2. Training	<code>PYTHONPATH=. python src/train.py</code>	<code>model_comparison.csv</code> , <code>final_model.joblib</code> , <code>tier_cutoffs.json</code>
3. Evaluation	<code>PYTHONPATH=. python src/evaluate.py</code>	<code>metrics.json</code> and diagnostic plots

Step	Command	Produces
4. App	PYTHONPATH=. streamlit run app/streamlit_app.py	Dashboard at localhost:8501

10. Project File Index

Path	Purpose
data/raw/	Original CSV files (not modified)
data/processed/sample_100k.csv	The 100,000-row sample used for modelling
notebooks/01_eda.ipynb, 02_modeling.ipynb	Exploratory analysis and model experiments
src/preprocess.py, train.py, evaluate.py	Preprocessing pipeline, training and comparison, test evaluation
models/	final_model.joblib, tier_cutoffs.json, metrics.json
reports/figures/, reports/results/	All plots and the model comparison table
app/streamlit_app.py	The web application
docs/notes.md, docs/viva_prep.md	Running notes and viva preparation